🚀 Guia Completo de Migração para Vercel

Controle de Efetivo de Bombeiros - Dashboard de Folgas Obrigatórias

---

📋 Índice

1. Pré-requisitos
2. SQL Schema Completo
3. Variáveis de Ambiente
4. Passo-a-Passo de Instalação
5. Estrutura do Projeto
6. Deploy na Vercel

🔧 Pré-requisitos

Você precisará de:

- **GitHub Account** (para versionamento)
- **Supabase Account** (para banco de dados)
- **Vercel Account** (para hospedagem)
- **Node.js 18+** (para desenvolvimento local)
- **Git** (para versionamento)

🗄️ SQL Schema Completo

Criar Tabelas no Supabase

Acesse seu projeto Supabase e execute este SQL no **SQL Editor**:

SQL

```sql
-- ==========================================
-- TABELA: BOMBEIROS
-- ==========================================
CREATE TABLE IF NOT EXISTS bombeiros (
```

```sql
  id VARCHAR(255) PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  equipe VARCHAR(10) NOT NULL CHECK (equipe IN ('VD', 'AM', 'AZ')),
  data_inicio TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Índice para melhor performance
CREATE INDEX idx_bombeiros_nome ON bombeiros(nome);
CREATE INDEX idx_bombeiros_data_inicio ON bombeiros(data_inicio);

-- ============================================
-- TABELA: ESCALAS
-- ============================================
CREATE TABLE IF NOT EXISTS escalas (
  id VARCHAR(255) PRIMARY KEY,
  bombero_id VARCHAR(255) NOT NULL,
  data VARCHAR(10) NOT NULL,
  sigla VARCHAR(10) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (bombero_id) REFERENCES bombeiros(id) ON DELETE CASCADE,
  UNIQUE KEY unique_bombero_data (bombero_id, data)
);

-- Índices para melhor performance
CREATE INDEX idx_escalas_bombero_id ON escalas(bombero_id);
CREATE INDEX idx_escalas_data ON escalas(data);
CREATE INDEX idx_escalas_bombero_data ON escalas(bombero_id, data);

-- ============================================
-- TABELA: USUÁRIOS (para autenticação futura)
-- ============================================
CREATE TABLE IF NOT EXISTS users (
  id VARCHAR(255) PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  nome VARCHAR(255) NOT NULL,
  role VARCHAR(50) DEFAULT 'user' CHECK (role IN ('admin', 'user')),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Índice para melhor performance
CREATE INDEX idx_users_email ON users(email);

-- ============================================
-- POLÍTICAS RLS (Row Level Security)
```

```sql
-- ==========================================

-- Habilitar RLS em todas as tabelas
ALTER TABLE bombeiros ENABLE ROW LEVEL SECURITY;
ALTER TABLE escalas ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Políticas para BOMBEIROS
CREATE POLICY "Bombeiros - SELECT para todos" ON bombeiros
  FOR SELECT USING (true);

CREATE POLICY "Bombeiros - INSERT para autenticados" ON bombeiros
  FOR INSERT WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Bombeiros - UPDATE para autenticados" ON bombeiros
  FOR UPDATE USING (auth.role() = 'authenticated');

CREATE POLICY "Bombeiros - DELETE para autenticados" ON bombeiros
  FOR DELETE USING (auth.role() = 'authenticated');

-- Políticas para ESCALAS
CREATE POLICY "Escalas - SELECT para todos" ON escalas
  FOR SELECT USING (true);

CREATE POLICY "Escalas - INSERT para autenticados" ON escalas
  FOR INSERT WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Escalas - UPDATE para autenticados" ON escalas
  FOR UPDATE USING (auth.role() = 'authenticated');

CREATE POLICY "Escalas - DELETE para autenticados" ON escalas
  FOR DELETE USING (auth.role() = 'authenticated');

-- Políticas para USERS
CREATE POLICY "Users - SELECT próprio perfil" ON users
  FOR SELECT USING (auth.uid() = id OR auth.role() = 'authenticated');

CREATE POLICY "Users - UPDATE próprio perfil" ON users
  FOR UPDATE USING (auth.uid() = id);
```

🔐 Variáveis de Ambiente

Arquivo `.env.local` (Desenvolvimento)

Bash

```
# ============================================
# SUPABASE CONFIGURATION
# ============================================
VITE_SUPABASE_URL=https://seu-projeto.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# ============================================
# DATABASE CONNECTION
# ============================================
DATABASE_URL=mysql://user:password@host:3306/database_name

# ============================================
# JWT SECRET (para sessões )
# ============================================
JWT_SECRET=sua-chave-secreta-super-segura-aqui-min-32-caracteres

# ============================================
# OAUTH (Manus OAuth - opcional)
# ============================================
VITE_APP_ID=seu-app-id
VITE_OAUTH_PORTAL_URL=https://seu-oauth-url
OAUTH_SERVER_URL=https://seu-oauth-server

# ============================================
# NODE ENVIRONMENT
# ============================================
NODE_ENV=development
```

Arquivo `.env.production` (Produção - Vercel)

Bash

```
# Mesmo que .env.local, mas com valores de produção
VITE_SUPABASE_URL=https://seu-projeto.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
DATABASE_URL=mysql://user:password@host:3306/database_name
JWT_SECRET=sua-chave-secreta-super-segura-aqui-min-32-caracteres
NODE_ENV=production
```

📦 Passo-a-Passo de Instalação

Passo 1: Clonar o Repositório

```bash
git clone https://github.com/seu-usuario/controle_efetivo_bombeiros.git
cd controle_efetivo_bombeiros
```

Passo 2: Instalar Dependências

```bash
pnpm install
# ou
npm install
```

Passo 3: Configurar Variáveis de Ambiente

```bash
# Copiar arquivo de exemplo
cp .env.example .env.local

# Editar com seus valores
nano .env.local  # ou use seu editor favorito
```

Passo 4: Criar Banco de Dados no Supabase

1. Acesse supabase.com
2. Crie um novo projeto
3. Vá para **SQL Editor**
4. Execute o SQL Schema fornecido acima
5. Copie as credenciais para `.env.local`

Passo 5: Testar Localmente

```bash
pnpm dev
```

Acesse: `http://localhost:5173`

Passo 6: Fazer Commit e Push

Bash

```bash
git add .
git commit -m "Configuração inicial para Vercel"
git push origin main
```

📁 Estrutura do Projeto

Plain Text

```
controle_efetivo_bombeiros/
├── client/                      # Frontend React
│   ├── src/
│   │   ├── pages/              # Páginas principais
│   │   ├── components/         # Componentes reutilizáveis
│   │   ├── contexts/           # React Contexts
│   │   ├── lib/                # Funções utilitárias
│   │   ├── App.tsx             # Componente raiz
│   │   └── main.tsx            # Entry point
│   ├── public/                 # Arquivos estáticos
│   └── index.html              # HTML principal
├── server/                      # Backend Node.js + tRPC
│   ├── routers.ts              # Definição de procedures tRPC
│   ├── db.ts                   # Queries do banco de dados
│   └── index.ts                # Entry point do servidor
├── drizzle/                     # Migrações do banco de dados
│   ├── schema.ts               # Definição de tabelas
│   └── migrations/             # Arquivos de migração
├── package.json                # Dependências do projeto
├── tsconfig.json               # Configuração TypeScript
├── vite.config.ts              # Configuração Vite
└── .env.local                  # Variáveis de ambiente (não commitar )
```

🚀 Deploy na Vercel

Passo 1: Conectar GitHub à Vercel

1. Acesse vercel.com
2. Clique em **"New Project"**

3. Selecione seu repositório GitHub
4. Clique em **"Import"**

Passo 2: Configurar Variáveis de Ambiente

Na página de configuração do projeto:

1. Vá para **"Environment Variables"**
2. Adicione cada variável de `.env.production`:
   - `VITE_SUPABASE_URL`
   - `VITE_SUPABASE_ANON_KEY`
   - `SUPABASE_SERVICE_ROLE_KEY`
   - `DATABASE_URL`
   - `JWT_SECRET`
   - `NODE_ENV` = `production`

Passo 3: Configurar Build

- **Framework Preset**: `Other`
- **Build Command**: `pnpm build`
- **Output Directory**: `dist`
- **Install Command**: `pnpm install`

Passo 4: Deploy

Clique em **"Deploy"** e aguarde a conclusão.

✅ Checklist de Verificação

Após o deploy, verifique:

- [ ] Página carrega sem erros
- [ ] Bombeiros aparecem na dashboard
- [ ] Consegue adicionar novo bombeiro
- [ ] Consegue marcar escala no calendário
- [ ] Consegue marcar FO
- [ ] Dados persistem após refresh

- [ ] Sem badge "Made with Manus"

🐛 Troubleshooting

Erro: "Cannot find module 'supabase'"

```bash
pnpm install @supabase/supabase-js
```

Erro: "DATABASE_URL is not defined"

Certifique-se de que a variável está configurada em **Vercel Settings > Environment Variables**

Erro: "CORS error"

Adicione sua URL de Vercel em **Supabase > Authentication > URL Configuration**

Dados não sincronizam

Verifique se `SUPABASE_SERVICE_ROLE_KEY` está correto

📞 Suporte

Para dúvidas ou problemas:

1. Verifique os logs em **Vercel Dashboard > Deployments**
2. Verifique os logs do Supabase em **Supabase Dashboard > Logs**
3. Teste localmente com `pnpm dev`

📄 Licença

MIT - Criado por Sgt Clovis

© 2026 Controle de Frequência e Afastamentos • 2º SGB • PB Andradina • by Sgt Clovis